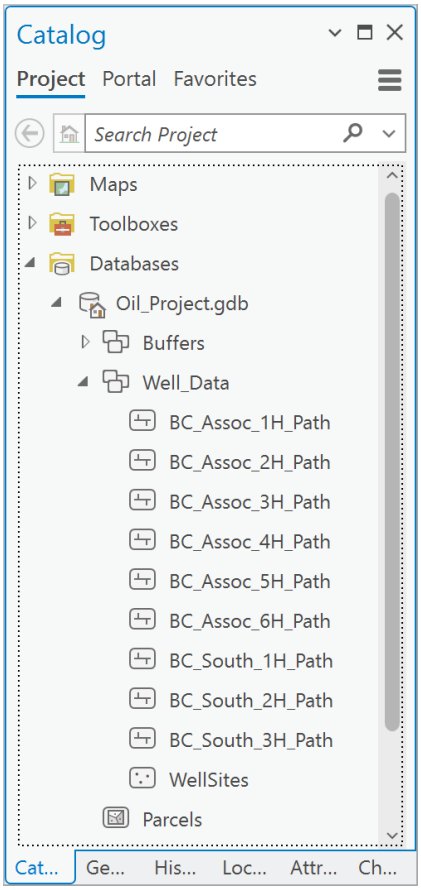
# Midterm Project

## Scenario

A gas well drilling company has made an application to drill several wells in Oleander. To perform this drilling, the company must notify and get a signed lease document from all the property owners that are located over the planned drill paths. The city's Engineering division has asked that you generate a set of lists of property owners for each drill path. Before each path is drilled, the city will hold a public meeting with those homeowners and the drilling company to work out any issues that might exist. Because you do not know when each well will go online, you will create a separate mailing list for each well path and have them on hand for use when they are needed. A mailing list will need to be a text file containing the addresses of the properties.

To create the lists, buffer each well path by 200 feet and select the parcels that intersect the buffer. Then export the attribute table of selected parcels to a csv file. Since you only want the addresses in the csv file that contains more than two dozen of fields, you will need to read the csv file, pull the addresses out, and write them in a new text file.

## Data




Download the Midterm.zip file, unzip it to a working folder, and open the project ("Midterm.aprx") to look at the data. Explore the default geodatabase Oil_project.gdb in the **Catalog pane**. The database contains a Parcels **feature class** and a Well_data **feature dataset** (a "folder" in the gdb) that contains nine well path **feature classes**. Therefore, your script will finally create nine mailing list files, one for each well path.

- The Parcels feature class has a Prop_add field that maintains the addresses of the properties. This is where you will get the property address info.
- When the attribute table of selected parcels is exported to a csv file, the Prop_add field is the 20th field in the csv file.

## Solution

Generally, you can use the arcpy ListFeatureClasses() function to get a list of well path (**polyline** feature class) names in the Well_Data feature dataset. Then you can iterate over the list and process each well path in a loop.

At each iteration, you have a well path (feature class name). To find properties over it, it takes a few steps. First, you can create a buffer around the well path using the Buffer() tool. Second, you will use the buffer to select parcels that intersect it. Selecting parcels can be accomplished by the SelectLayerByLocation() tool (in the management module). Please read the online documentation of this tool and pay attention to the first parameter of the tool (in_layer). The first parameter takes a feature layer object as input. Please note

that a feature layer is not a feature class. Therefore, prior to calling the SelectLayerByLocation() tool, you must use the MakeFeatureLayer() tool (in the management module) to create a feature layer object from the Parcels feature class. Third, the SelectLayerByLocation() tool returns a **result object** that contains selected parcel features. You can use the CopyRows() tool (in the management module) with the result object as an input to export a csv file. The csv file will contain attributes of the selected parcels. Check the ArcGIS documentation for how to use the CopyRows() tool. Finally, you can open the csv file and read its contents line by line (using another loop). Values in a line are separated by commas. Once you read a line, get the address from it, and write the address in an output text file.

In your program, you should name the output mailing list files to reflect the well path names in this format: mailing_list_<well path name>.txt in which <well path name> is the feature class name of a well path. For example, if the well path feature class name is *BC_Assoc_1H_Path*, the mailing list file name should be *mailing_list_BC_Assoc_1H_Path.txt*. Use string concatenation to create dynamic file names.

Once a mailing list file is created, a well path has been successfully processed, and this marks the end of one iteration. This process will repeat nine times since there are nine well paths.

**Note:**

1) Carefully read the online documentation of each key geoprocessing tool, especially the MakeFeatureLayer_management(), SelectLayerByLocation_management(), and CopyRows() tools which are not demonstrated in the textbook. Make sure you understand the parameters and output of these tools.

2) Use the Python console to test or practice necessary arcpy functions and tools. You may manually go through the process with one well path using the Python console.

3) When you call the Buffer_analysis() tool, make sure to save the resulting buffer feature class in the Buffers feature dataset in the geodatabase. Use string concatenation skills to dynamically name the output buffer feature class in the format <well path name>_buffer. For example, if the well path feature class is "BC_Assoc_3H_Path", the buffer feature class should be named "BC_Assoc_3H_Path_buffer", and the output name in the geodatabase should be "Buffers/ BC_Assoc_3H_Path_buffer".

4) Note that if a geoprocessing tool produces a new data set (e.g., a feature class), the **result object** returned from the tool carries the output dataset path, which can be used as an input of subsequent tools. For example, if you use variable *well_buffer* to hold the result object of the buffer tool

   well_buffer = arcpy.Buffer_analysis(…)

   The result object *well_buffer* can be passed to the SelectLayerByLocation_management() tool as an (input) argument.

   parcels_selected = arcpy.SelectLayerByLocation_management(…, …, well_buffer)

   Similarly, you can pass the result object *parcels_selected* to the CopyRows () tool as an input.

5) Refer to Lecture 5 and Lab 4 for reading and writing text files.

6) Make sure to set a workspace environment so that you do not have to use full paths of the data.

7) Set environment arcpy.env.overwriteOutput = True. When you repeatedly test your script, existing data in the geodatabase can prevent a tool from producing an output with the same name as an existing data, thus cause the script fail to run. Setting this environment can usually solve the problem. If this still does not work, you need to manually delete the existing output data before a new test.

8) Geoprocessing tool arcpy.management.Delete() may be used to delete intermediate data in scripts.

**Submission:**

- Name your Python script midterm.py.
- Document your script and adequately comment your code.

**Grading:**

- Workability (50 points): Your script should run and produce desired results.
- Code efficiency and robustness (30 points): Your script should use proper Python programming techniques to effectively carry out tasks, for example, it should use loops instead of copying and pasting a block of code to carry out iterative tasks. It should have adequate flexibility and reusability, for example, it uses variables to hold values instead of hard-coding values in the script. Your script should have the capability of handling possible exceptions.
- Readability and coding style (20 points): Your script should be readable (e.g. it uses proper variable names and functional blocks are separated by blank lines). It should be well documented and contains adequate comments. It should display user-friendly messages when running.

# Project Flowchart

